

Bioinformatics and Systems Biology Summer School 2026

<https://summeratmasaryk.cz/genes-in-motion-bioinformatics-and-systems-biology>

Genes in Motion

Exact Methods for Inferring Boolean Models of GRNs

David Šafránek

Faculty of Informatics, Masaryk University, Brno

Modelling with Incomplete Knowledge

Consistent models comply with experimental measurements as much as possible

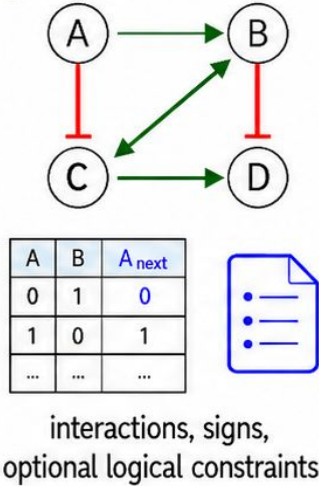
Consistent models can explain consequences of gene regulation and they can even predict new insights worth of further experimentation

Credibility of models is never guaranteed but should improve with increasing knowledge

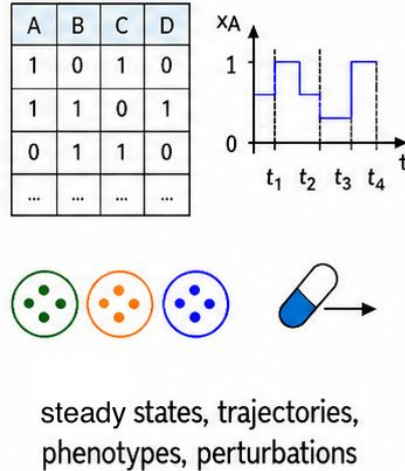
How to obtain consistent and credible models?

Boolean Network Inference

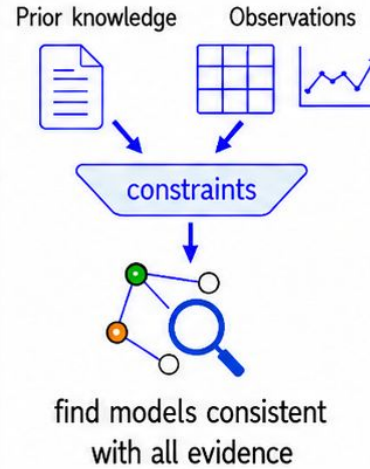
1 Prior knowledge



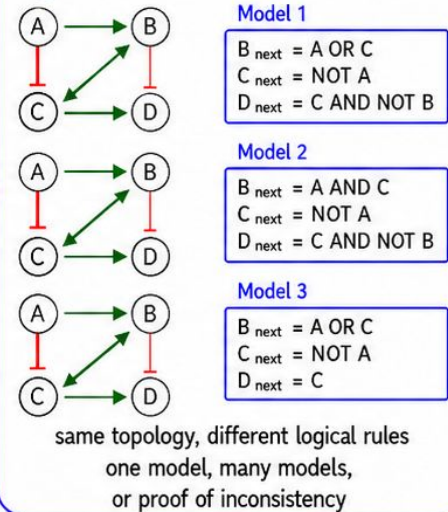
2 Observations



3 Inference problem



4 Candidate models



Input Domain – Prior Knowledge

The input for BN model inference is typically a GRN (regulation graph):

- annotated genes and predicted regulations
- some regulations can be assigned a sign (encoding **monotonicity**)
- not all regulations need to be required to take their effect (**essentiality**)
- some update functions can be partially given

(in that case we use the so-called Partially Specified BNs)

Technically, the input is a huge ensemble of BNs built over the given GRN topology

Constraining the Domain

Local properties of regulations directly impose respective update functions

- they form **static constraints**

Experimental data and literature-based knowledge provide (incomplete) information on how the biological system behaves in time (incl. long term)

- these observations form **dynamic constraints**

All constraints are put together to restrict the domain space of candidate models

Solution Domain – Candidate Models

Satisfiability approach

- Checking for existence of a model

- Negative answer supported with a proof

 - e.g., contradictory constraints

Enumeration approach

- Iterating through the output domain (typically very large)

- Random sampling

Inference Tools

Biodivine Sketchbook

<https://github.com/sybila/biodivine-sketchbook>

- based on Partially Specified Boolean Networks
- temporal logic is used to capture dynamic constraints
- symbolic model checking methods allow to represent the solution domain



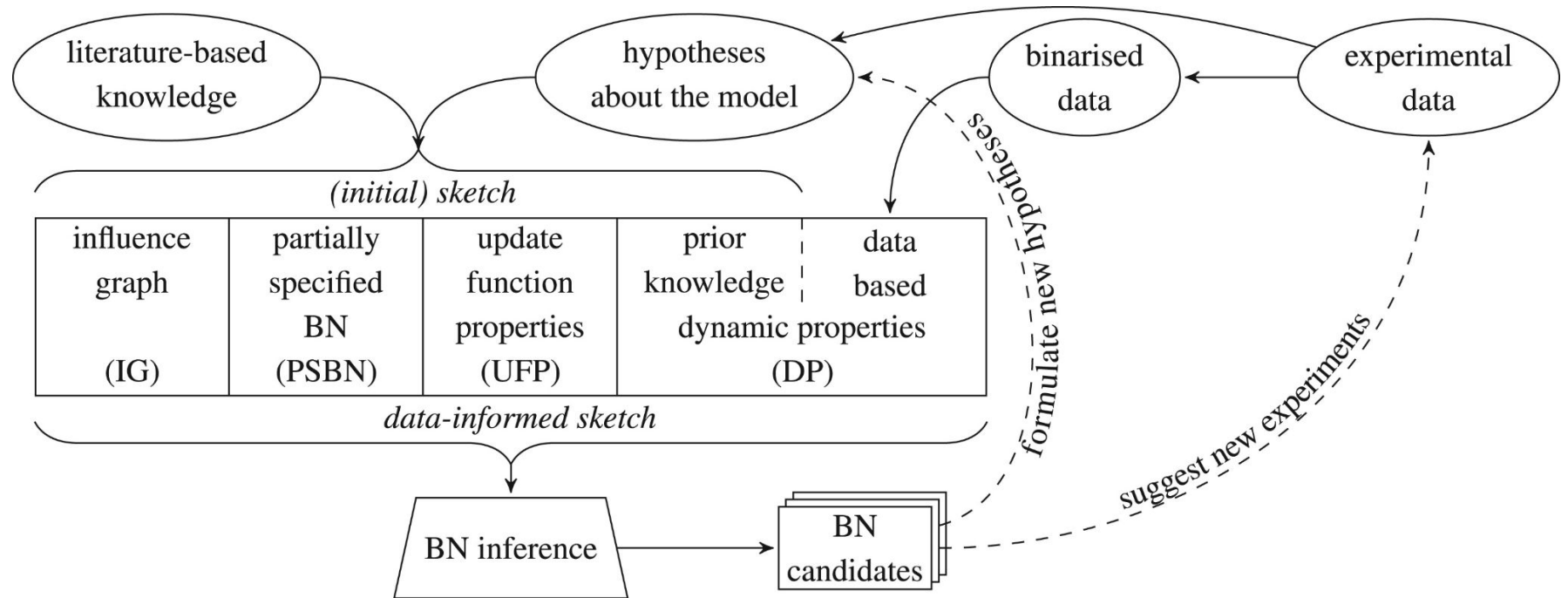
BoNesis

<https://bnediction.github.io/bonesis/index.html>

- based on rule-based reasoning / logical programming
- represents the solution domain as a set of models satisfying a logical specification capturing the constraints, technically restricted to monotonous update functions
- uses Answer Set Programming as the underlying inference method

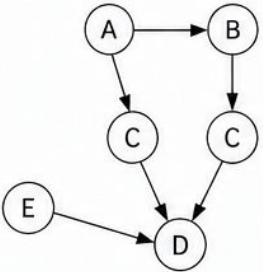


Inference Tools: Sketchbook's View



Inference Tools: BoNesis's View

1. Network topology (known or hypothesized)



2. Observations / data

-  **steady states**
(e.g., $A=1, D=0, \dots$)
-  **trajectories**
(time-series, transitions)
-  **perturbations**
(knock-outs/ins, inputs)
-  **additional constraints**
(mutual exclus., bounds, ...)

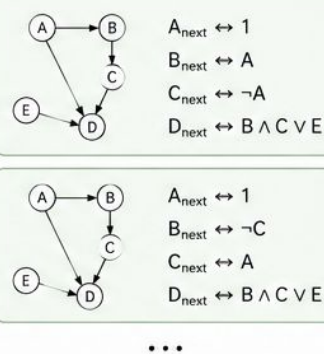
3. ASP encoding (according to MP semantics)



Encode specification as Answer Set Program

- topology (structure constraints)
- observations / constraints
- dynamics (Majority / MP semantics)

4. Solution space (all models satisfying the specifications)



5. Explore / sample / analyse models (enumerate and inspect solutions)



enumerate / sample
Boolean networks



inspect alternative
logic rules



analyse dynamics
(attractors, reachability,
robustness, perturbations)

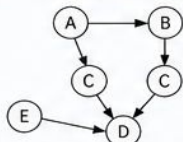
...

BoNesis Inference Workflow

INPUT: specification of the search problem

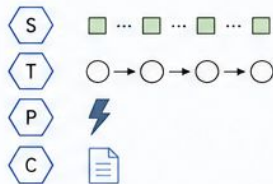
Topology

- directed graph over variables



Observations

- steady states
- trajectories (transitions)
- perturbations
- additional constraints



Other options

- e.g., bounds on node activity, mutual exclusions, priorities, ...

BONESIS: ASP inference engine

Encoding to Answer Set Programming
(facts and constraints)

Answer Set Solver
(compute stable models)



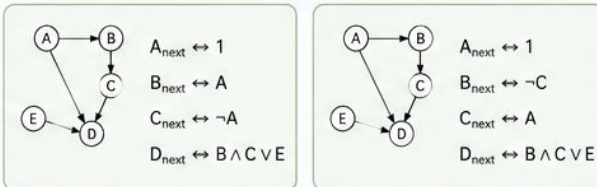
Model Enumeration
(all Boolean networks satisfying
the specification)

Analyse models

OUTPUT: inferred models

Solution space

- set of all Boolean networks consistent with the specification



Explore / sample / analyse models

- enumerate models, inspect rules, analyse dynamics (attractors, reachability, robustness, perturbations, ...)



Answer Set Programming – Basic Idea

Declare what must hold; the solver searches for self-consistent answer sets.

01 LOGIC PROGRAM

Facts + rules

```
bird(tweety).  
  
flies(X) :- bird(X),  
           not  
           abnormal(X).  
  
not = default negation
```

02 GROUND + SOLVE

Create candidates

$X = \text{tweety}$

1. Ground variables
2. Apply supported rules
3. Enforce constraints
4. Test stability

03 STABLE MODEL

Answer set

```
{ bird(tweety),  
  flies(tweety) }
```

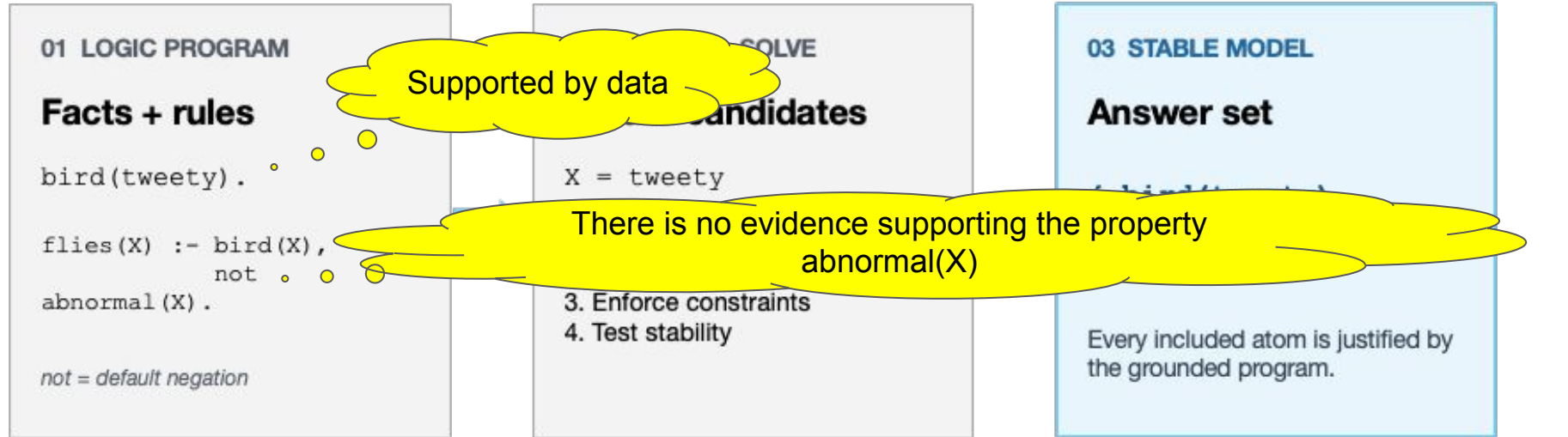
Every included atom is justified by the grounded program.

WHY THE RULE FIRES

bird(tweety) is a fact + abnormal(tweety) is not derivable $\rightarrow$ flies(tweety) enters the model

Answer Set Programming – Basic Idea

Declare what must hold; the solver searches for self-consistent answer sets.

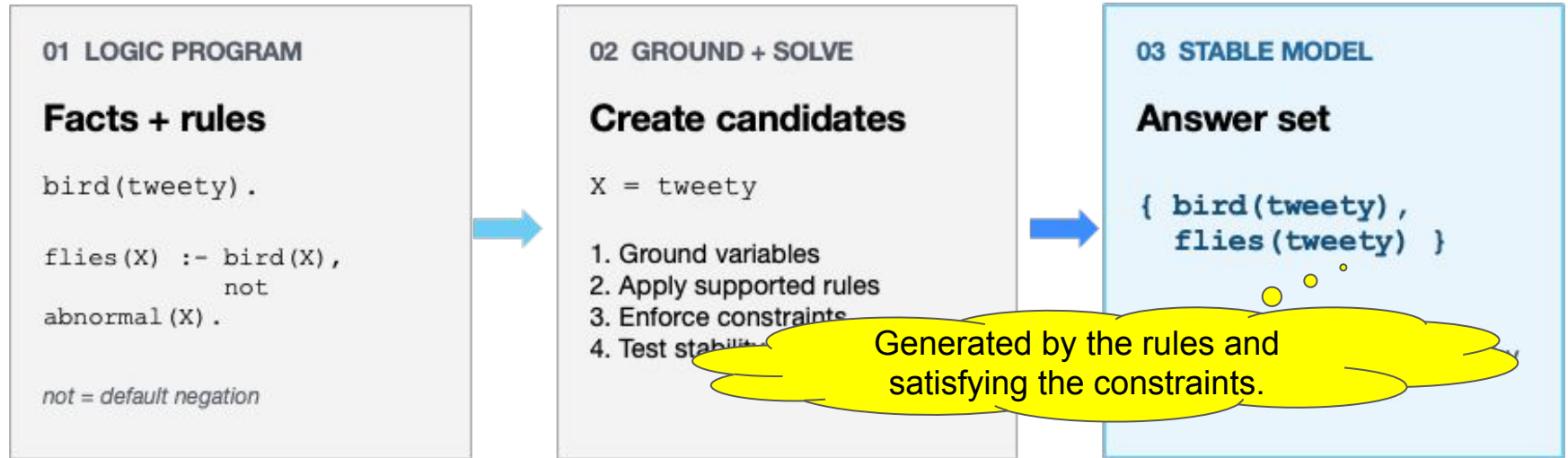


WHY THE RULE FIRES

bird(tweety) is a fact + abnormal(tweety) is not derivable → flies(tweety) enters the model

Answer Set Programming – Basic Idea

Declare what must hold; the solver searches for self-consistent answer sets.



WHY THE RULE FIRES

bird(tweety) is a fact + abnormal(tweety) is not derivable → flies(tweety) enters the model

Answer Set Programming – Encoding BN Inference

01 BONESIS INPUT

Model domain + data

```
dom =  
InfluenceGraph.complete(["A","B","C"])  
  
data = {  
  "init":  {"A":0, "B":0, "C":0},  
  "marker": {"A":1, "C":0}  
}
```

Defines the candidate space of locally-monotone Boolean networks and two partial observations.

02 DECLARATIVE SPECIFICATION

Constrain the dynamics

```
x = ~bo.obs("init")  
y = ~bo.obs("marker")  
x >= bo.fixed(y)  
with bo.mutant({"C":0}):  
  x / y
```

BoNesis compiles these predicates into ASP: generate candidate Boolean functions, enforce MP reachability and mutation constraints, then ask ASP solver for stable models.

03 ANSWER SET SOLUTION

Satisfying Boolean networks (Answer Sets)

```
A <- A | (!B&C)  
B <- !B | (A&!C)  
C <- !B
```

Each stable ASP model encodes one BN satisfying all declared properties.

From Data to Constraints

Discretisation is part of the modelling assumption.

Different thresholds or marker choices may lead to different inferred model spaces.

1. scRNA-seq data

cell × gene expression matrix
noisy snapshots of single-cell states

2. Normalisation + batch correction

remove technical variation
make cells/samples comparable

3. Gene selection

marker genes, transcription factors,
pathway genes, known regulators

4. Discretisation

quantitative expression
→ Boolean / multi-valued levels

5. Partial observations

cluster A: {Stem=1, Diff=0}
cluster B: {Stem=0, Diff=1}

6. BoNesis constraints

fixed points, reachability,
non-reachability, perturbation effects

From Data to Constraints

Discretisation is part of the modelling assumption.

Different thresholds or marker choices may lead to different inferred model spaces.

Input values

one gene, many cells
not yet discretised

expression



cells

preprocessed expression
for selected gene X

K-means partition

k = 2 groups expression values
by numerical similarity



low centroid

cells assigned to
nearest centroid

Qualitative encoding

order centroids, then assign levels

lower centroid $\rightarrow X = 0$

higher centroid $\rightarrow X = 1$

cell	expr.	X
c ₁	0.2	0
c ₂	3.1	1

Static Constraints – Monotonicity

Monotonicity of a regulation implies an impact on the regulated gene's dynamics:

- positive – regulated gene's expression is expected to be increased in case the expression of the regulator is increased
- negative – regulated gene's expression is expected to be decreased in case the regulator's expression is increased

In BoNesis we expect all inferred functions to be monotonous.

Encoding BN Inference – Observations

- an observation represents an abstract state
 - we expect the observations to be partial (not all variables observed)

O = **obs** ({**a**:0, **b**:1}) ... represents an abstract state where a==0 and b==1

C = ~**O** ... refers to a (default) state meeting the observation

Encoding BN Inference – State Constraints

- an observation represents an abstract state
 - we expect the observations to be partial (not all variables observed)

$C \neq C'$... states C and C' must be two different states

$C[X] == 0 \mid 1$... variable X in state C is 0 (resp. 1)

$C[X] == C'[Y]$... variable X in state C is equal to variable Y in state C'

$C[X] \neq C'[Y]$... variable X in state C must have different value than variable Y in state C'

$C \neq \circ$... refers to a state not meeting the observation

Dynamic Constraints – Fixed Points and Trap Spaces

fixed(C) ... state C is a fixed point

in_attractor(C) ... there exist an attractor to which state C belongs

– a subspace of STG with no outgoing transitions is called a **trap space**

fixed(O) ... there exists a trap space where observation O is fixed

all_fixedpoints({O, O', ...})

... all fixed points must meet at least one of the listed observations

Dynamic Constraints – Reachability

`C` $\geq$ `C'` ... state `C'` is reachable from state `C` (most permissive semantics)

`C` $\geq$ `O` ... a state meeting observation `O` is reachable from state `C`

`C` $\geq$ `fixed(O)` ... a state in a trap space defined by `O` is reachable from state `C`

“ $\geq$ ” can be replaced with “/” denoting non-reachability

Operators can be composed:

`C` $\geq$ `C'` $\geq$ `fixed(C'' )`

... state `C''` is a fixed point which is reached from state `C` through state `C'`

Contexts – Perturbations

```
with mutant ({X:0, Y:1}) :
```

```
...
```

- properties in the context of the mutant block are understood under (permanent) perturbation of the variables X and Y

Hands On

Run the jupyter notebook supporting the lecture: [02-boolean-network-inference.ipynb](#)

Installation instructions for the corresponding Python environment are given in README (see the github folder <https://github.com/sybila/bioinformatics-summer-school/tree/main/bn>)